El Despacho — Análisis de arquitectura y hoja de ruta

Nov 3, 2026 · @veneko

Veredicto

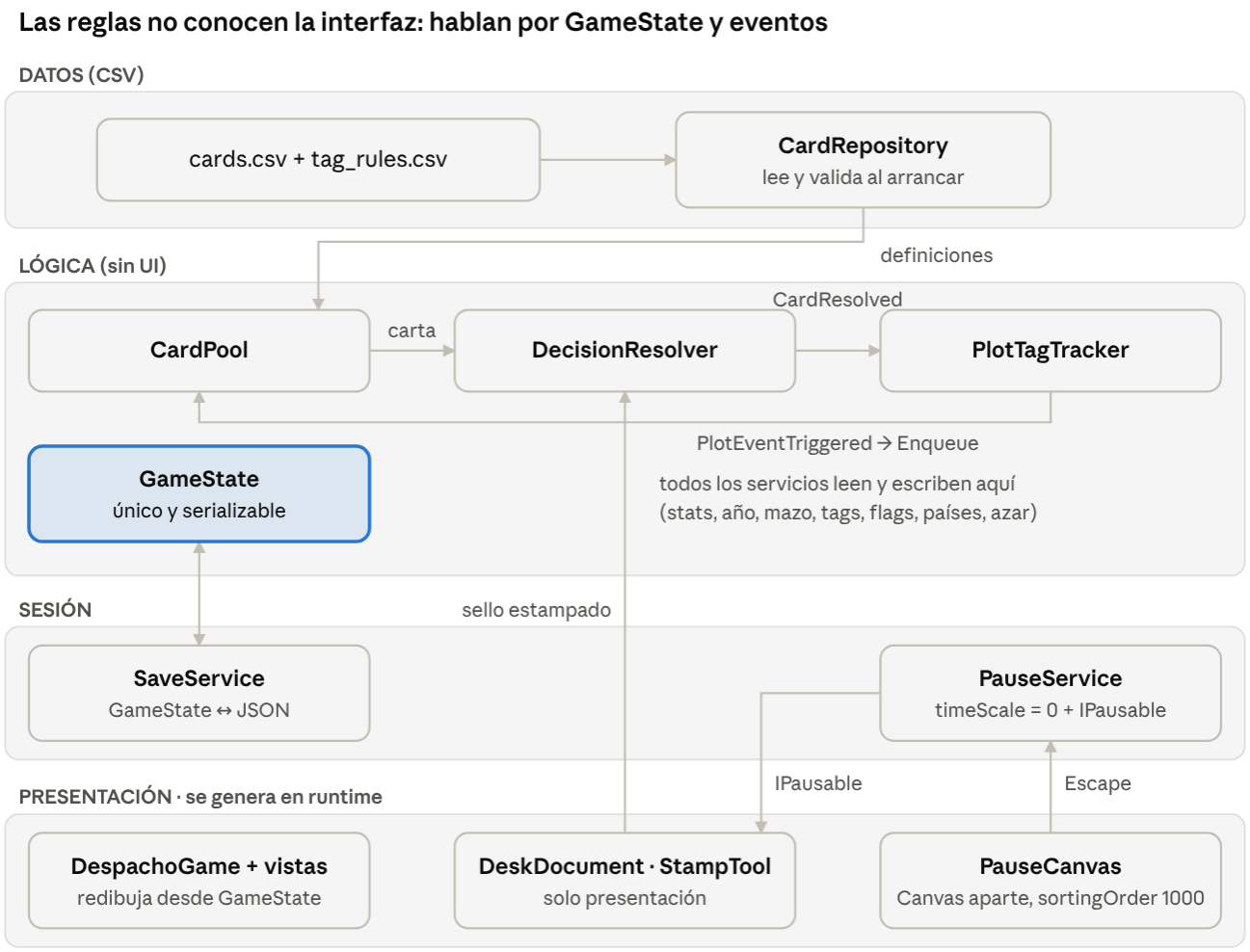
Los cuatro sistemas son viables sobre la arquitectura actual, pero no encima de ella tal cual: antes hay que sacar el estado de la partida de `DespachoGame.cs`, que hoy tiene 1.264 líneas y mezcla estado, reglas y construcción de la interfaz. Con ese paso previo, cada sistema se convierte en un módulo pequeño que no conoce la interfaz.

Sistema	Viabilidad	Riesgo principal
1. Cartas en CSV + CardPool	Alta	Parser propio (Unity no trae lector de CSV) y textos con comas, comillas y saltos de línea
2. Tags de trama	Alta	Definir bien qué cuenta (total o seguidas) y qué pasa si salta un evento con la mesa ya vacía
3. Menú de pausa	Media-alta	Las animaciones de expediente, sellos y temblor usan tiempo no escalado: `Time.timeScale = 0` **no** las para
4. Guardado y carga	Alta	Lo que se pide guardar no basta para reconstruir la partida: faltan países, flags, cola de cartas, noticias y el estado del azar

Lo que juega a favor: la interfaz se redibuja entera en cada acción (`Clear()` y `Show…()`). Eso hace que cargar una partida sea trivial: basta con restaurar el estado y llamar a la pantalla que toca. `DeskDocument` y `StampTool` ya son solo presentación y se comunican por callbacks, así que se pueden conservar.

Orden recomendado: extraer el estado → CSV y CardPool → tags de trama → guardado y carga → menú de pausa. La pausa va al final porque sus botones de Guardar y Cargar necesitan que exista el sistema de guardado.

Arquitectura objetivo



arquitectura objetivo · 4 capas

Cuatro capas, cada una en su `asmdef`: la lógica no puede referenciar la interfaz, y la interfaz solo llama a servicios y lee `GameState`. Los servicios no se llaman entre sí directamente salvo el flujo principal del mazo: se avisan con eventos de C# en una clase estática `GameEvents` (`CardResolved`, `PlotEventTriggered`, `StateChanged`, `PauseChanged`). Así, añadir o quitar el sistema de tags no obliga a tocar `CardPool` ni la interfaz.

Módulo	Responsabilidad	No hace
`CardRepository`	Leer y validar los CSV; dar `CardDefinition` inmutables por id	Decidir qué carta sale
`CardPool`	Estados del mazo, cola con prioridades, robo ponderado	Aplicar efectos

Módulo	Responsabilidad	No hace
`DecisionResolver`	Aplicar efectos, flags y seguimientos; publicar `CardResolved`	Contar tags
`PlotTagTracker`	Contadores por tag, evaluar reglas, publicar `PlotEventTriggered`	Tocar el mazo directamente
`GameState`	Todo el estado de la partida, serializable	Lógica
`SaveService`	`GameState` ↔ JSON, versiones, escritura segura	Decidir cuándo se puede guardar (eso lo dice la sesión)
`PauseService`	Tiempo, audio y avisar a los `IPausable`	Dibujar el menú
Vistas, `DeskDocument`, `StampTool`, `PauseCanvas`	Dibujar y recoger la entrada del jugador	Cambiar el estado sin pasar por un servicio

1. Motor de datos CSV y CardPool

Viable y recomendable: las 27 cartas de `Cards.cs` ya son datos puros (textos, números, ids), así que pasan a CSV sin perder nada.

Dónde va el archivo

- **`Assets/Resources/Data/cards.csv`** como `TextAsset`, leído con `Resources.Load<TextAsset>`. Es la opción más simple y viaja dentro de la build.
- Alternativa: `StreamingAssets/` si queréis editar cartas sin recompilar la build. Para un proyecto de clase no compensa.
- **Codificación UTF-8** (hay tildes, ñ, « »). Si se edita en Excel, guardar como «CSV UTF-8».
- **Separador `;`**: el Excel en español usa punto y coma por defecto, y los textos llevan muchas comas. Campos con `;`, comillas o saltos de línea van entre comillas dobles, con `""` para una comilla literal.

Cabecera exacta de `cards.csv`

```
id;tipo;personaje;cargo;color;texto;etiqueta_si;etiqueta_no;si_estabilidad;si_seguridad;si_prosperidad;no_estabilidad;no_seguridad;no_prosperidad;resultado_si;resultado_no;anio_min;anio_max;requiere;flag_si;flag_no;siguiente_si;siguiente_no;solo_encadenada;tags;peso;notas
huelga;normal;Tomás Ruiz;Líder del sindicato minero;#a0603a;"Los mineros llevamos tres semanas en huelga. Pedimos un 10 % más de sueldo. ¿Nos lodas?";;;6;0;-5;-5;0;0;Los mineros vuelven al trabajo cantando.;Ruiz sale del despacho dando un portazo.;1;3;;;;;huelga_general;0;#trama_sindicatos;1;
sabotaje_ev;evento_negativo;«Búho»;Jefe de los servicios secretos;#3a3a4a;"Una cadena de apagones paraliza el país. Alguien ha saboteado la red eléctrica.";ASUMIR;OCULTAR;-8;-6;-6;-14;-4;-4;...;...;1;4;;;;;;1;;1;Evento por tag
```

Columna	Tipo	Uso
`id`	texto único	Clave de la carta. Lo que guarda la partida y lo que referencian `siguiente_*` y las reglas de tags
`tipo`	`normal` / `evento_positivo` / `evento_negativo`	Solo para presentación (color del expediente, sonido) y filtros. No cambia reglas
`personaje`, `cargo`, `color`	texto, texto, `#rrggbb`	Cabecera y retrato del expediente
`texto`	texto	El problema
`etiqueta_si`, `etiqueta_no`	texto o vacío	Vacío = APROBADO / DENEGADO en el sello
`si_*`, `no_*`	6 enteros	Efectos sobre Estabilidad, Seguridad y Prosperidad
`resultado_si`, `resultado_no`	texto	Lo que se lee tras sellar
`anio_min`, `anio_max`	1–4	Ventana en la que puede salir
`requiere`	flag o vacío	Solo sale si existe ese flag (incluye los `ally_<país>` / `cut_<país>` de la diplomacia)
`flag_si`, `flag_no`	flag o vacío	Flag que deja cada respuesta

Columna	Tipo	Uso
`siguiente_si`, `siguiente_no`	id o vacío	Carta que entra en la cola justo después
`solo_encadenada`	0/1	1 = nunca sale al azar (seguimientos y eventos)
`tags`	lista con `|`	Tags de trama: `#trama_sabotaje|#trama_militar`. Ver punto 2
`peso`	entero ≥ 1	Probabilidad relativa al robar (hoy todas valen 1)
`notas`	texto	Para diseño; el juego la ignora

El CardPool: estados de cada carta

El mazo deja de ser «una lista y un `HashSet` de usadas» dentro de `DespachoGame` y pasa a ser un servicio `CardPool` que conoce el estado de cada carta:

Estado	Significa	Pasa a…
Bloqueada	No cumple año o `requiere` todavía	Disponible cuando se cumple
Disponible	Puede salir al azar	Robada
Encolada	Forzada por `siguiente_*` o por un tag	Robada (con prioridad)
Robada	Está en la mesa ahora	Resuelta
Resuelta	Ya se selló (guarda si fue SÍ o NO)	—
Descartada	Pasó su `anio_max` sin salir	—

El estado «Bloqueada / Disponible» no se guarda: se calcula cada vez a partir del año y los flags. Solo se guardan las cartas Encoladas, la Robada, las Resueltas con su respuesta y las Descartadas.

API mínima (sin dependencias de UI):

- `CardDefinition Draw(GameState s)`: primero la cola, luego azar ponderado por `peso` entre las Disponibles, con prioridad a las que tienen `requiere` (como hoy).
- `void Enqueue(string id, QueuePriority p)`: lo usan los seguimientos y el sistema de tags.

- `void Resolve(string id, bool yes)` y `IEnumerable<CardDefinition> Available(GameState s)`.

Validación al cargar. El lector debe comprobar y avisar en consola si hay `id` repetidos, referencias a ids que no existen (`siguiente_*`, reglas de tags), años fuera de 1–4 o colores mal escritos. Un error en una hoja de cálculo no debe convertirse en una carta que nunca sale sin que nadie se entere.

2. Tags de trama (ID compartido)

Viable, y encaja con lo que ya existe: hoy las cartas se encadenan con `siguiente_*` y `flags`; los tags añaden una tercera vía, por acumulación. La clave es separar **qué cartas comparten tag** (en `cards.csv`) de **qué pasa cuando se acumula** (en un segundo archivo de reglas). Así una misma trama puede tener a la vez un castigo y una recompensa, y se ajustan umbrales sin tocar las cartas.

En `cards.csv`: columna `tags`

- Una carta puede tener **varios tags** separados por `|`: `#trama_sabotaje|#trama_militar`. Cada respuesta suma en todos.
- Prefijo `#` obligatorio y minúsculas, para que el validador detecte errores de escritura (`#Trama_Sabotaje` y `#trama_sabotaje` serían tramas distintas).
- Las cartas de evento llevan `solo_encadenada = 1` para que nunca salgan al azar, y normalmente sin tags, para que no se realimenten.

Archivo nuevo: `tag_rules.csv`

```
id_regla;tag;contador;umbral;carta_evento;una_vez;prioridad;reiniciar;notas
sabotaje_fatal;#trama_sabotaje;no;3;sabotaje_ev;1;alta;0;3 veces mirando a otro lado ante sabotajes
sabotaje_premio;#trama_sabotaje;si;3;sabotaje_premio_ev;1;alta;0;3 veces actuando contra el sabotaje
sindicatos_paz;#trama_sindicatos;neto_si;2;pacto_social_ev;1;normal;1;Diferencia SÍ-NO de +2
```

Columna	Valores	Uso
`id_regla`	texto único	Lo que se guarda para saber que ya saltó
`tag`	`#...`	Trama que vigila

Columna	Valores	Uso
`contador`	`si` / `no` / `total` / `neto_si` / `neto_no` / `racha_si` / `racha_no`	Qué se cuenta. `neto_si` = SÍ menos NO. `racha_*` = respuestas iguales seguidas dentro de esa trama
`umbral`	entero	Cuándo salta (≥)
`carta_evento`	id de `cards.csv`	Carta que se inyecta
`una_vez`	0/1	1 = solo puede saltar una vez por partida
`prioridad`	`alta` / `normal`	`alta` = delante de la cola (antes que un seguimiento); `normal` = al final de la cola
`reiniciar`	0/1	1 = pone a cero los contadores del tag al saltar (permite que vuelva a saltar si `una_vez = 0`)

Lógica

Un servicio `PlotTagTracker` guarda, por cada tag, `si`, `no` y la racha actual, y la lista de reglas ya disparadas. No conoce la interfaz ni el mazo; solo escucha y avisa:

1. El jugador sella. `DecisionResolver` aplica efectos y flags y publica el evento `CardResolved(cardId, yes)`.
2. `PlotTagTracker` lo recibe, suma en los contadores de cada tag de esa carta y evalúa solo las reglas de esos tags.
3. Si una regla cumple su umbral y no está disparada, publica `PlotEventTriggered(reglaId, cartaEventoId, prioridad)` y la marca como disparada.
4. `CardPool` lo recibe y llama a `Enqueue(cartaEventoId, prioridad)`. La carta sale en el siguiente robo, saltándose el azar.

Casos límite que hay que decidir y documentar:

- **Mesa ya vacía.** Si el evento salta con la última carta del año, hoy `CardsDoneThisYear` exige la cola vacía, así que el evento saldría igualmente antes de poder cerrar el año. Recomendado: que los eventos **no cuenten** para el límite de 5 asuntos por año (hoy cualquier carta incrementa `cardsThisYear`).
- **Dos reglas a la vez.** Se encolan las dos por prioridad; si empatan, por orden del archivo.

- **Error Fatal.** Si un evento debe poder terminar la partida, mejor una columna `fin_partida` en `cards.csv` con el id de un final que meter números enormes en los efectos.
- **Visibilidad.** Conviene que el jugador intuya la trama: el asesor o la gaceta pueden comentar «ya van dos sabotajes sin respuesta» cuando un contador está a uno del umbral.

3. Menú de pausa con Escape

Viable, pero es el punto con más trampas, porque la pausa toca entrada, tiempo, interfaz y guardado a la vez.

Problemas concretos con el código actual

- **`Time.timeScale = 0` no detiene nada visible.** `DeskDocument`, `StampTool` y `Wiggle` animan con `Time.unscaledDeltaTime`, y el bloqueo de doble clic usa `Time.unscaledTime`. Se hizo así a propósito, para que funcionen aunque el tiempo esté parado, pero significa que en pausa el expediente seguiría moviéndose y el botón del mapa temblando.
- **`Clear()` borra todo lo que cuelga del Canvas principal.** Si el menú de pausa se crea dentro de ese Canvas, desaparece en cuanto el juego redibuja una pantalla.
- **La interfaz de debajo sigue recibiendo clics.** Sin una capa que los bloquee, se podría sellar un expediente con el menú abierto.
- **Arrastre a medias.** Si se pulsa Escape con un sello o un expediente cogido, quedan en un estado intermedio.
- **Solo Input System nuevo.** El proyecto tiene `activeInputHandler = 1`: `Input.GetKeyDown(KeyCode.Escape)` lanza una excepción. Hay que usar `Keyboard.current.escapeKey.wasPressedThisFrame` o, mejor, la acción `UI/Cancel` que ya existe en `InputSystem_Actions` (así funciona también con mando).
- **«Volver al menú principal» no puede recargar la escena.** `RuntimeInitializeOnLoadMethod(AfterSceneLoad)` se ejecuta **una sola vez** al darle a Play, no en cada carga de escena. Si se hace `SceneManager.LoadScene`, el objeto del juego se destruye y no se vuelve a crear: pantalla vacía.

Diseño propuesto

- **Un Canvas propio para la pausa** (`PauseCanvas`, `sortingOrder = 1000`), hermano del Canvas del juego y no hijo de `root`, así `Clear()` no lo toca. Se crea una vez al arrancar y se activa o desactiva.

- **Fondo a pantalla completa** con `raycastTarget = true` y un oscurecido semitransparente: bloquea los clics a la mesa.
- **`PauseService`** con `Pause()` / `Resume()` y un evento `PauseChanged(bool)`. Al pausar: `Time.timeScale = 0`, `AudioListener.pause = true` y aviso a todo lo que implemente `IPausable`.
- **`IPausable`** en `DeskDocument`, `StampTool` y `Wiggle`: al pausar cancelan el arrastre en curso (como ya hacen en `OnApplicationFocus`) y congelan su animación. Es un cambio pequeño: las tres ya tienen la lógica de «volver a su sitio».
- **Botones:** Continuar → `Resume()`. Opciones → un subpanel en el mismo Canvas (volumen y pantalla completa en `PlayerPrefs`, separados de la partida). Guardar / Cargar → `SaveService` (punto 4). Menú principal → `GameSession.ReturnToTitle()`, que reinicia el estado en memoria y llama a `ShowTitle()`, sin recargar la escena.
- **Cuándo no se abre:** en la pantalla de título, en un final y durante la animación de archivo de un expediente.

4. Guardado y carga en JSON

Viable y, gracias a que la interfaz se redibuja desde el estado, la carga es sencilla: restaurar los datos y llamar a `ShowOffice()`. El problema está en **qué** se guarda: con solo estadísticas, año, mazo y contadores de tags, al cargar se perderían las decisiones diplomáticas, los flags, las consecuencias retrasadas y la cola de cartas, y la partida cambiaría.

Qué tiene que llevar el archivo

Bloque	Contenido	Pedido
Cabecera	`version` del formato, fecha, versión del juego	No, pero imprescindible para no romper partidas al cambiar el formato
Estadísticas	Estabilidad, Seguridad, Prosperidad	Sí
Año	Año actual y asuntos atendidos ese año	Sí
Mazo	Carta en la mesa, cola, resueltas con su respuesta, descartadas	Sí
Tags de trama	Contadores por tag y reglas ya disparadas	Sí

Bloque	Contenido	Pedido
Flags	Todos los flags activos	**Falta**
Países	Por país: decisión, atacado, relación tensa o agradecida	**Falta**
Consecuencias	Eventos retrasados pendientes y efectos que duran varios turnos	**Falta**
Contadores de final	Alianzas, ataques, sanciones	**Falta**
Azar	Semilla y posición del generador	**Falta**; ver abajo
Presentación	Expediente abierto o no, sellado o no y dónde (posición y giro del sello), noticias	**Falta**


```
{
  "version": 1,
  "savedAt": "2026-11-03T13:05:00",
  "stats": { "estabilidad": 54, "seguridad": 61, "prosperidad": 47 },
  "year": 2,
  "cardsThisYear": 3,
  "pool": {
    "current": "corrupcion",
    "queue": ["corrupcion_2"],
    "resolved": [ { "id": "huelga", "yes": false }, { "id": "tanques", "yes":
true } ],
    "discarded": ["estatua"]
  },
  "plotTags": {
    "counters": [ { "tag": "#trama_sabotaje", "si": 1, "no": 2, "racha": -2 }
],
    "firedRules": []
  },
  "flags": ["espionaje", "ally_lindmark"],
  "countries": [ { "id": "tarvessa", "decision": "Alianza", "attacked":
false, "annoyed": false, "grateful": false } ],
  "pendingEvents": [ { "countryId": "korovia", "year": 3, "which": "ally" }
],
  "drains": [ { "reason": "sin los suministros de Tarvessa", "prosperidad":
-2, "turns": 2 } ],
```

```
  "rng": { "seed": 918273, "state": 4412907713 },
  "desk": { "documentOpened": true, "answered": false, "stamp": null },
  "news": ["..."]
}
```

Decisiones técnicas

- **Guardar ids, nunca objetos.** Las cartas y los países se guardan por `id`; las definiciones salen siempre de los CSV y de `CountryDatabase`. Los eventos retrasados se guardan como «país + cuál (`ally` / `cut`)», no copiando su texto.
- **`JsonUtility` en vez de Newtonsoft.** No añade paquetes (ya tuvimos problemas con `Packages/`), pero no serializa `Dictionary` ni `HashSet`: el DTO usa listas de pares, como en el ejemplo. Se convierte al estado de juego y de vuelta con dos funciones `ToSave()` / `FromSave()`.
- **El azar actual no se puede guardar.** `System.Random` no expone su estado. Hay que sustituirlo por un generador propio pequeño (xorshift, unas 15 líneas) con semilla y estado. Así, tras cargar, las cartas siguientes son las mismas que habrían salido.
- **Puntos seguros.** Se guarda en el despacho y en el mapa; no a mitad de una visita diplomática, ni durante una animación del expediente o con un sello cogido. El estado lógico (`documentOpened`, `cardAnswered`, sello) sí se guarda: `DeskDocument.Initialize(alreadyOpen)` ya sabe reconstruir un expediente abierto.
- **Archivo.** En `Application.persistentDataPath/saves/slot1.json`. Se escribe primero en `slot1.json.tmp` y luego se reemplaza, para no corromper la partida si el juego se cierra a mitad. Las opciones van aparte, en `PlayerPrefs`.
- **Versiones.** Si `version` no coincide, `SaveService` migra o rechaza con un mensaje claro, en vez de cargar datos a medias.

Cuellos de botella de la arquitectura actual

El mayor freno es `DespachoGame.cs`: casi todo lo demás se resuelve en cuanto el estado sale de ahí.

Problema	Dónde	A qué sistema afecta	Solución
Clase «Dios» de 1.264 líneas: estado, reglas, mazo, diplomacia y las 8 pantallas juntas	`DespachoGame.cs`	Todos	Extraer `GameState` (datos puros) y servicios; `DespachoGame` queda como orquestador y las pantallas como vistas

Problema	Dónde	A qué sistema afecta	Solución
Estado repartido en campos privados sueltos (`estabilidad`, `usedCards`, `flags`, `pending`, `drains`…)	`DespachoGame.cs`	Guardado	Un único `GameState` serializable: lo que no esté ahí no existe
`System.Random` sin estado accesible	`DespachoGame.cs`	Guardado	Generador propio con semilla y estado
Animaciones con tiempo no escalado	`DeskDocument`, `StampTool`, `Wiggle`, `cardAnsweredAt`	Pausa	Interfaz `IPausable` + cancelar arrastres al pausar
`Clear()` destruye todo bajo `root`	`DespachoGame.Clear`	Pausa	Canvas de pausa separado, fuera de `root`
Cartas definidas en código y con `Color` de Unity	`Cards.cs`	CSV	`CardDefinition` inmutable con color en hex; `Cards.cs` solo como respaldo hasta validar el CSV
Cualquier carta consume uno de los 5 asuntos del año	`DrawCard`	Tags	Contar solo las cartas normales; seguimientos y eventos son extra
La prueba de Maximo lee campos privados por reflexión (`deskDocument`, `documentOpened`, `estabilidad`…)	`DeskValidationProbe.cs`	Todos	Al refactorizar se rompe: actualizarla en el mismo paso, o darle una API de pruebas
Todo compila en `Assembly-CSharp`	Proyecto	Todos	`asmdef` por capa (ver arquitectura): impide que la lógica dependa de la UI sin querer
Volver al título no puede recargar la escena	`RuntimeInitializeOnLoadMethod`	Pausa	Reinicio en memoria (`GameSession.ReturnToTitle`)

Sobre `DeskDocument` y los sellos en concreto: no son un problema de diseño. Ya son solo presentación, se comunican por callbacks (`onOpened`, `onStamped`) y saben reconstruirse abiertos. Solo necesitan dos cosas: implementar `IPausable` y que el estado que hoy les pasa `DespachoGame` (`documentOpened`, posición del sello…) venga de `GameState`.

Hoja de ruta por fases

Cada fase es un commit (o rama) que deja el juego funcionando igual que antes más lo nuevo. Ninguna fase depende de que la siguiente esté hecha.

Fase	Objetivo	Entregables	Terminada cuando…
0. Cimientos	Sacar el estado de `DespachoGame` sin cambiar el juego	`GameState`, `GameRng`, `GameEvents`, `asmdef` de Core / Game / Presentation / Tests; prueba de Maximo actualizada	Se juega igual, 0 errores y la prueba da 32/32
1. CSV	Cartas fuera del código	`CsvReader` (`;` , comillas, UTF-8), `CardDefinition`, `CardRepository` + validador; `cards.csv` exportado automáticamente desde `Cards.cs` con un script de editor (sin reescribir a mano)	Una prueba compara las 27 cartas del CSV con las de `Cards.cs` campo a campo y coinciden; luego se borra `Cards.cs`
2. CardPool	Mazo como servicio	`CardPool` con estados, cola con prioridades y robo ponderado; `DecisionResolver` publicando `CardResolved`	`DespachoGame` ya no toca `usedCards` ni `cardQueue`; cadenas como la huelga siguen funcionando
3. Tags de trama	Eventos por acumulación	Columna `tags`, `tag_rules.csv`, `PlotTagTracker`, 2 cartas de evento de ejemplo	Responder 3 veces NO a cartas con `#trama_sabotaje` inyecta el evento como siguiente carta, una sola vez
4. Guardado	Persistencia completa	`SaveData` (DTO con `version`), `SaveService` con escritura segura, `ToSave` / `FromSave`	Guardar a mitad del año 2, cerrar, cargar y obtener el mismo estado y **las mismas cartas siguientes**

Fase	Objetivo	Entregables	Terminada cuando…
5. Pausa	Menú con Escape	`PauseCanvas`, `PauseService`, `IPausable` en las 3 animaciones, Opciones en `PlayerPrefs`, `GameSession.ReturnToTitle`	Con un sello cogido, Escape lo suelta y lo para todo; no se puede sellar con el menú abierto; Guardar / Cargar / Menú principal funcionan

Documentación en cada fase: comentario de cabecera en cada clase (qué hace, qué no, de quién depende), entrada en `CHANGELOG.md` siguiendo el formato de la v0.3.1 y un `Documentacion/Datos_CSV.md` que explique columnas y reglas para quien diseñe cartas sin programar.

Reparto posible: cerrada la fase 0, las fases 1–3 (datos y reglas) y la 5 (pausa) tocan zonas distintas y pueden avanzar en paralelo. La pausa puede empezar con Guardar y Cargar desactivados y conectarlos cuando termine la fase 4, que a su vez necesita la 2 y la 3.

Decisiones que tiene que tomar el equipo

Ninguna bloquea la fase 0; conviene cerrarlas antes de la fase en la que aparecen. Entre paréntesis, la propuesta de este análisis.

- [ ] **Fase 1 — Separador del CSV:** `;` o `,` (propuesta: `;`, por Excel en español).
- [ ] **Fase 1 — Dónde viven los datos:** `Resources` o `StreamingAssets` (propuesta: `Resources`).
- [ ] **Fase 3 — «3 respuestas negativas»:** ¿en total o seguidas? (propuesta: el CSV admite las dos con `contador`; por defecto, en total).
- [ ] **Fase 3 — ¿Los eventos y seguimientos cuentan para los 5 asuntos del año?** (propuesta: no).
- [ ] **Fase 3 — ¿Un Error Fatal puede terminar la partida?** Si sí, añadir la columna `fin_partida`.
- [ ] **Fase 4 — Ranuras de guardado:** una o varias (propuesta: 3 ranuras + autoguardado al empezar cada año).
- [ ] **Fase 4 — ¿Guardar en cualquier momento o solo en puntos seguros?** (propuesta: despacho y mapa; no durante visitas ni animaciones).
- [ ] **Fase 5 — Opciones:** qué entra (propuesta: volumen general, pantalla completa y resolución).